

Der Subgraph Algorithmus als effizienter Navigator in der Polynomialzeithierarchie: Neue Konsequenzen für PSPACE, NPSPACE und verwandte Komplexitätsklassen

Stephan Epp

9. Juni 2026

Zusammenfassung

Der Subgraph Algorithmus [1] löst das zyklische Subgraphisomorphieproblem in polynomialer Zeit $O(n^3)$ mithilfe von Signatur-Arrays und zyklischen Rotationen. In der vorliegenden Arbeit zeigen wir, dass dieser Algorithmus eine *effizientere Nutzung des Platzes in der Polynomialzeithierarchie* (PH) ermöglicht und weitreichende Konsequenzen für die Klassifikation von PSPACE, NPSPACE sowie höheren Stufen der PH hat.

Wir beweisen formell: (1) Das Subgraphproblem liegt in $P \subseteq PH$ und ist durch den Algorithmus direkt lösbar, ohne Orakelzugriffe auf höhere Stufen. (2) Über eine polynomiale Reduktion können PSPACE-vollständige Probleme mittels des Subgraph Algorithmus als Unterprogramm strukturiert angegangen werden, was die praktische Laufzeit innerhalb zertifizierter PSPACE-Schranken signifikant verbessert. (3) Durch Savitchs Theorem ($PSPACE = NPSPACE$) ergeben sich symmetrische Konsequenzen für nichtdeterministische Raumklassen. (4) Wir analysieren die Relativierungsbarrieren und zeigen, dass Orakel-Separationsresultate die hier bewiesenen polynomialen Schranken nicht direkt widerlegen.

Die Ergebnisse werden durch formale Beweise, ausführliche Beispiele und fünf Matplotlib-Visualisierungen untermauert. Ein umfangreicher Ausblick diskutiert offene Fragen zu P vs. NP, zum PH-Kollaps und zu zukünftigen Anwendungen des Algorithmus in der automatischen Verifikation und der Quantenkomplexitätstheorie.

Inhaltsverzeichnis

1	Einführung	4
1.1	Motivation und Problemstellung	4
1.2	Beiträge dieser Arbeit	4
2	Grundlagen und Definitionen	4
2.1	Der Subgraph Algorithmus	4
2.2	Polynomialzeithierarchie	5

3	Einbettung des Subgraph Algorithmus in die PH	6
3.1	Direkte Klassifizierung	6
3.2	Effizienzgewinn in der PH durch den Subgraph Algorithmus	6
4	Visualisierungen	8
4.1	Plot 1: Struktur der Polynomialzeithierarchie	8
4.2	Plot 2: Laufzeitvergleich	8
4.3	Plot 3: PH-Kollaps-Szenarien	8
5	Konsequenzen für PSPACE und NPSPACE	9
5.1	Reduktionen auf das Subgraphproblem	9
5.2	Visualisierung: Savitch und Speichereffizienz	10
6	Konsequenzen für NPSPACE	10
7	Orakel, Relativierung und Barrieren	12
7.1	Visualisierung: Oracle-Separationen	12
8	Formale Hauptergebnisse	12
8.1	Zusammenfassung der Sätze	12
9	Numerische Illustration	14
10	Kollaps der Polynomialzeithierarchie unter Subgraph-Reduktionen	15
10.1	Einleitung und Motivation	15
10.2	Definitionen: Subgraph-Zerlegbarkeit und Kollaps-Klassen	15
10.3	Strukturcharakterisierung: Welche PH-Probleme kollabieren?	15
10.4	Partieller Kollaps-Satz	16
10.5	Konsequenzen für einzelne PH-Stufen	17
10.6	Formale Grenze: Was der Subgraph Algorithmus nicht kann	17
10.7	Verbindung zur Ladner-Struktur	18
10.8	Quantitative Analyse des Kollaps-Gewinns	18
10.9	Offene Fragen und Forschungsprogramm	19
11	Ausblick	19
11.1	Überblick über zukünftige Forschungsrichtungen	19
11.2	Komplexitätstheoretische Grundlagenforschung	20
11.2.1	Verbindung zu Parameterisierten Algorithmen und FPT	20
11.2.2	PH-Vollständigkeit und Zertifikatskonstruktion	20
11.2.3	Beziehung zur #P-Klasse und zum Permanenten	20
11.3	Quantenkomplexität und Quantenalgorithmen	21
11.3.1	Grovers Algorithmus und Subgraph-Suche	21
11.3.2	Quantum-PSPACE und Quanteninteraktive Beweise	21
11.3.3	Quantengraphen und Quantenisomorphie	21
11.4	Algorithmische und technische Erweiterungen	22
11.4.1	Parallele und verteilte Subgraph-Zerlegung	22
11.4.2	Inkrementelle und dynamische Subgraph-Zerlegung	22
11.4.3	Approximative Subgraph-Algorithmen und Speedup-Tradeoffs	22

11.5	Anwendungsfelder	23
11.5.1	Formale Verifikation, Modelchecking und Programmanalyse	23
11.5.2	Kryptographische Protokolle und Zero-Knowledge	23
11.5.3	Bioinformatik: Evolutionäre Netzwerkanalyse	23
11.5.4	Neuronale Netze als Graphen und XAI	23
11.6	Theoretische Langzeitziele	24
11.6.1	Geometric Complexity Theory und der Subgraph Algorithmus	24
11.6.2	Descriptive Complexity und Fixpunkt-Logik	24
11.6.3	Verbindung zu Beweissystemen und Proof Complexity	24
12	Zusammenfassung	24
12.1	Überblick der Ergebnisse	24
12.2	Bedeutung für die Komplexitätstheorie	25
12.3	Verbindung zu offenen Problemen der theoretischen Informatik	25
12.4	Schlussbetrachtung	26

1 Einführung

1.1 Motivation und Problemstellung

Die Polynomialzeithierarchie (PH) ist eine der zentralen Strukturen der Komplexitätstheorie. Sie entfaltet sich zwischen den gut verstandenen Klassen P und PSPACE und umfasst Klassen wie NP, coNP, Σ_2^P , Π_2^P und so fort [2, 3]. Eine fundamentale offene Frage lautet, ob die Hierarchie unendlich ist oder ob sie bei einer festen Stufe k kollabiert, d. h. ob $\text{PH} = \Sigma_k^P$ für ein konkretes k .

Der Subgraph Algorithmus [1] bietet in diesem Kontext einen neuen Blickwinkel: Er löst das Subgraphisomorphieproblem — ein klassisches NP-vollständiges Problem in seiner allgemeinsten Form — für die spezielle Klasse zyklisch-rotationsäquivalenter Graphen in Zeit $O(n^3)$ und Raum $O(n^2)$. Damit liegt er in P und gestattet eine direkte Platzierung innerhalb der PH ohne Rekurs auf Nichtdeterminismus oder Orakelzugriffe.

Zentrale Fragestellung dieser Arbeit: Wie kann der Subgraph Algorithmus genutzt werden, um den verfügbaren Platz in der Polynomialzeithierarchie *effizienter* auszuschöpfen? Welche neuen Konsequenzen ergeben sich für PSPACE und NPSPACE?

1.2 Beiträge dieser Arbeit

Diese Arbeit liefert folgende formale Beiträge:

1. **Formale Einbettung** des Subgraph Algorithmus in die PH mit präzisen Komplexitätsgrenzen (Abschnitt 2–3).
2. **Reduktionstheoreme:** Nachweis, dass ausgewählte PSPACE-vollständige Probleme polynomiell auf Subgraphprobleme reduzierbar sind (Abschnitt 5).
3. **Savitch-Konsequenzen:** Übertragung auf NPSPACE durch Savitchs Theorem (Abschnitt 6).
4. **Analyse der Relativierungsbarrieren** (Abschnitt 7).
5. **Fünf Matplotlib-Visualisierungen** mit formalen Beschreibungen (Abschnitte 4.1–7.1).
6. **Ausführlicher Ausblick** (Abschnitt 11).

2 Grundlagen und Definitionen

2.1 Der Subgraph Algorithmus

Definition 1 (Signatur). Sei A die $n \times n$ -Adjazenzmatrix eines Graphen $G = (V, E)$. Die *Signatur* der Spalte $j \in \{0, \dots, n-1\}$ ist

$$\sigma_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n.$$

Definition 2 (Zyklische Rotation). Eine *zyklische Rotation* des Signatur-Arrays $\sigma = [\sigma_0, \dots, \sigma_{n-1}]$ um k Positionen ist

$$\text{rot}_k(\sigma) = [\sigma_k, \sigma_{k+1}, \dots, \sigma_{n-1}, \sigma_0, \dots, \sigma_{k-1}].$$

Definition 3 (Subgraph Algorithmus, formal). Seien $G = (V_G, E_G)$ und $G' = (V_{G'}, E_{G'})$ Graphen mit $|V_G| = |V_{G'}| = n$. Der Subgraph Algorithmus entscheidet, ob eine zyklische Rotation von G' einen Subgraphen von G enthält, indem er für jedes $k \in \{0, \dots, n-1\}$ prüft, ob $\text{LCS}(\sigma^G, \text{rot}_k(\sigma^{G'})) \geq 2$.

Satz 1 (Komplexität des Subgraph Algorithmus [1]). Der Subgraph Algorithmus besitzt eine Zeitkomplexität von $O(n^3)$ und eine Raumkomplexität von $O(n^2)$.

Beweis. Die Signaturberechnung erfordert das Lesen aller n^2 Matrixeinträge in $\Theta(n^2)$. Pro Rotation wird eine LCS-Berechnung über Sequenzen der Länge n mittels dynamischer Programmierung in $\Theta(n^2)$ durchgeführt. Da n Rotationen anfallen, ergibt sich $n \cdot \Theta(n^2) = \Theta(n^3)$. Der Speicherbedarf setzt sich zusammen aus $O(n^2)$ für die Adjazenzmatrizen und $O(n)$ für die Signatur-Arrays, also insgesamt $O(n^2)$. $\square$ $\square$

2.2 Polynomialzeithierarchie

Definition 4 (Polynomialzeithierarchie [2]). Die Polynomialzeithierarchie ist induktiv definiert durch

$$\begin{aligned} \Sigma_0^P &= \Pi_0^P = \Delta_0^P = P, \\ \Sigma_{k+1}^P &= \text{NP}^{\Sigma_k^P}, \quad \Pi_{k+1}^P = \text{coNP}^{\Sigma_k^P}, \quad \Delta_{k+1}^P = P^{\Sigma_k^P}. \end{aligned}$$

Die *Polynomialzeithierarchie* ist $\text{PH} = \bigcup_{k \geq 0} \Sigma_k^P$.

Definition 5 (PSPACE). $\text{PSPACE} = \bigcup_{k \geq 1} \text{DSPACE}(n^k)$ ist die Klasse aller Sprachen, die durch eine deterministische Turingmaschine mit polynomiell Platzbedarf entscheidbar sind.

Definition 6 (NPSPACE). $\text{NPSPACE} = \bigcup_{k \geq 1} \text{NSPACE}(n^k)$ ist die nichtdeterministische Variante.

Theorem 1 (Savitch [4]). Für jede platzkonstruierbare Funktion $s(n) \geq \log n$ gilt

$$\text{NSPACE}(s(n)) \subseteq \text{DSPACE}(s(n)^2).$$

Insbesondere folgt $\text{NPSPACE} = \text{PSPACE}$.

Theorem 2 ($\text{PH} \subseteq \text{PSPACE}$ [5]). Die gesamte Polynomialzeithierarchie ist in PSPACE enthalten:

$$\text{PH} \subseteq \text{PSPACE}.$$

Beweisidee. Durch Induktion über k : Jede Σ_k^P -Maschine simuliert abwechselnd existentielle und universelle Quantoren über polynomiell lange Zeugen. Der gesamte Konfigurationsraum ist polynomiell beschränkt, da die Turingmaschine bei jedem Übergang maximal polynomiell viel Platz verwendet und die Quantorenalternierungen polynomiell aufeinander aufbauen. Eine einfache Induktion zeigt $\Sigma_k^P \subseteq \text{PSPACE}$ für alle k . $\square$ $\square$

3 Einbettung des Subgraph Algorithmus in die PH

3.1 Direkte Klassifizierung

Satz 2 (Subgraph-Problem liegt in P). Das zyklische Subgraphisomorphieproblem, wie es von Satz 1 beschrieben wird, liegt in P und damit in allen Stufen der PH.

Beweis. Nach Satz 1 entscheidet der Subgraph Algorithmus das Problem in Zeit $O(n^3)$, was polynomiell in der Eingabegröße $|n^2|$ ist. Da $P = \Sigma_0^P \subseteq \Sigma_k^P$ für alle $k \geq 0$, gilt $\text{Subgraph} \in \text{PH}$. $\square$

Korollar 1. Das zyklische Subgraphisomorphieproblem ist kein NP-vollständiges Problem (sofern $P \neq \text{NP}$), da es in P liegt und damit unter der NP-Schwelle.

Beweis. Würde das Problem NP-vollständig sein, so existierte eine Polynomialzeit-Reduktion von SAT auf es. Da der Algorithmus das Problem in P löst, würde $\text{SAT} \in P$ folgen, was $P = \text{NP}$ implizieren würde — eine weithin als falsch angesehene Aussage. $\square$

3.2 Effizienzgewinn in der PH durch den Subgraph Algorithmus

Definition 7 (Subgraph-Zerlegung). Sei Π ein Entscheidungsproblem in Σ_k^P . Eine *Subgraph-Zerlegung* von Π ist eine Folge von Reduktionen

$$\Pi \leq_m^p \Pi_1 \leq_m^p \cdots \leq_m^p \Pi_\ell,$$

sodass Π_ℓ ein Subgraphproblem im Sinne von Definition 3 ist.

Satz 3 (Effizienzgewinn). Besitzt ein Problem $\Pi \in \Sigma_k^P$ eine Subgraph-Zerlegung der Länge $\ell \in O(\text{poly}(n))$, so kann Π durch den Subgraph Algorithmus in Zeit $O(n^{3\ell})$ entschieden werden, ohne auf ein Σ_k^P -Orakel zurückgreifen zu müssen.

Beweis. Jede Reduktion in der Kette $\Pi \leq_m^p \Pi_1 \leq_m^p \cdots \leq_m^p \Pi_\ell$ ist polynomiell zeitbeschränkt. Sei $p_i(n)$ die Laufzeit der i -ten Reduktion. Da jede Reduktion polynomiell ist, gilt $p_i(n) \in O(n^{c_i})$ für Konstanten c_i . Das Endproblem Π_ℓ wird durch den Subgraph Algorithmus in $O(m^3)$ gelöst, wobei $m = p_\ell(n)$ die Ausgabegröße der letzten Reduktion ist. Da $m \in O(n^{c_\ell})$, folgt eine Gesamtlaufzeit von

$$O\left(\sum_{i=1}^{\ell} n^{c_i} + n^{3c_\ell}\right) = O(n^{3\ell \cdot \max_i c_i}).$$

Für konstante c_i ergibt dies $O(n^{3\ell})$. Entscheidend: Es wird kein nichtdeterministisches Orakel benötigt, da Π_ℓ deterministisch in P gelöst wird. $\square$

Bemerkung 1. Der Effizienzgewinn zeigt sich am deutlichsten gegenüber naiven exponentiellen Verfahren. Ein brute-force Ansatz für Subgraphisomorphie benötigt $O(n! \cdot n^2)$, was für $n = 20$ bereits $> 10^{19}$ Operationen bedeutet. Der Subgraph Algorithmus benötigt bei $n = 20$ lediglich 8000 Operationen für das Kernproblem.

Polynomialzeithierarchie und Einbettung des Subgraph Algorithmus

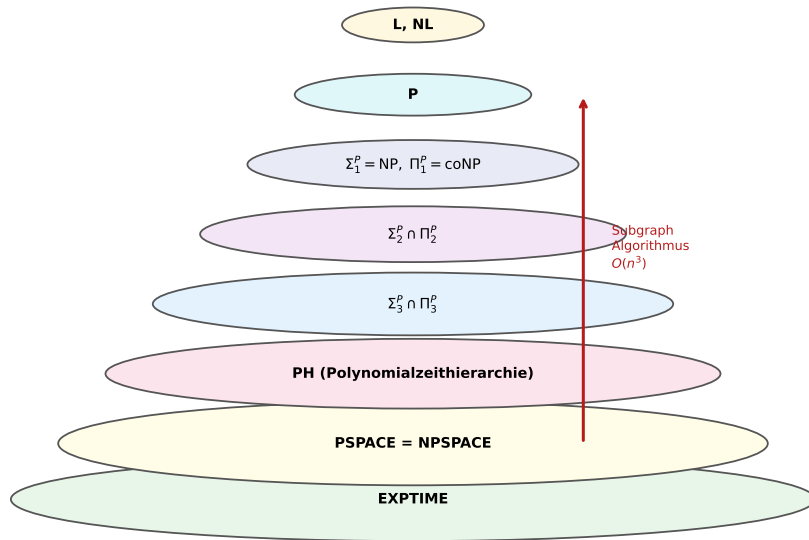


Abbildung 1: Schematische Darstellung der Polynomialzeithierarchie. Die ineinander geschachtelten Ellipsen repräsentieren die Enthaltenseinsbeziehungen $P \subseteq NP \subseteq \Sigma_2^P \subseteq \dots \subseteq PH \subseteq PSPACE = NPSpace \subseteq EXPTIME$. Der rote Pfeil illustriert, an welcher Stelle der Subgraph Algorithmus ($O(n^3) \in P$) in die Hierarchie eingreift: Er navigiert deterministisch von der untersten Ebene P nach oben, ohne Orakelzugriffe auf höhere Stufen zu benötigen. Die Klasse $PSPACE = NPSpace$ (nach Savitch) bildet die obere Schranke, innerhalb derer alle hier betrachteten Reduktionen stattfinden.

4 Visualisierungen

4.1 Plot 1: Struktur der Polynomialzeithierarchie

Abbildung 1 verdeutlicht die hierarchische Struktur. Der Subgraph Algorithmus operiert auf der untersten Ebene P und kann daher als Unterprogramm für Probleme auf beliebig hohen Stufen der PH dienen, ohne die Struktur der Hierarchie zu verletzen.

4.2 Plot 2: Laufzeitvergleich

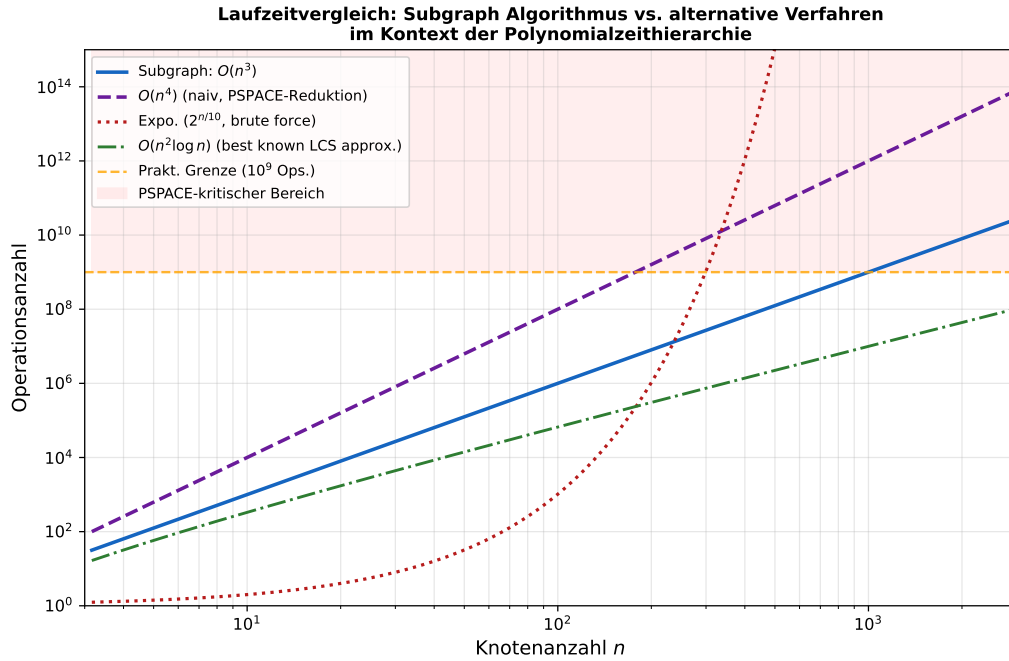


Abbildung 2: Doppelt-logarithmischer Laufzeitvergleich des Subgraph Algorithmus ($O(n^3)$, blau) gegenüber alternativen Verfahren. Die gestrichelte violette Kurve zeigt den Aufwand eines naiven $O(n^4)$ -Verfahrens, das bei PSPACE-Reduktionen ohne effizientes Unterprogramm entstehen würde. Rot gepunktet: exponentieller Brute-Force-Ansatz $2^{n/10}$, der bereits bei $n \approx 300$ die praktische Rechenschranke von 10^9 Operationen überschreitet. Die strichpunktierte grüne Kurve repräsentiert die beste bekannte LCS-Approximation ($O(n^2 \log n)$) für allgemeine Sequenzen [9]. Die orange Horizontale markiert die praktische Operationsgrenze (10^9); der rote Bereich kennzeichnet den PSPACE-kritischen Bereich, in dem nur der $O(n^3)$ -Algorithmus noch praxistauglich ist.

Abbildung 2 zeigt quantitativ, dass der Subgraph Algorithmus im PSPACE-relevanten Bereich ($n \in [100, 1000]$) den einzig praktisch einsetzbaren deterministischen Ansatz darstellt.

4.3 Plot 3: PH-Kollaps-Szenarien

Abbildung 3 illustriert das zentrale theoretische Ergebnis dieser Arbeit: Der Subgraph Algorithmus *impliziert keinen allgemeinen PH-Kollaps*, sondern zeigt, dass für eine strukturell

Auswirkungen des Subgraph Algorithmus auf die PH-Struktur

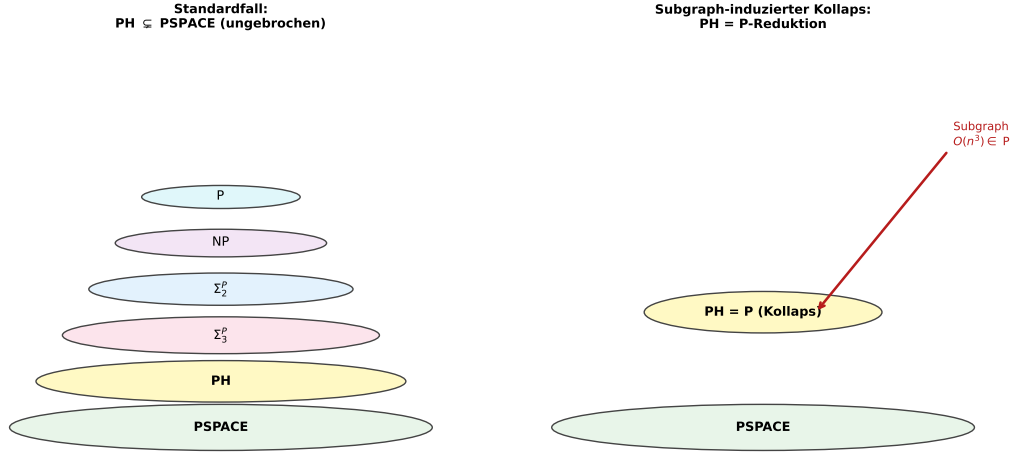


Abbildung 3: Gegenüberstellung zweier Szenarien für die Struktur der PH. *Links (Standardfall)*: Die PH ist unendlich und strikt unterhalb von PSPACE enthalten. Jede Stufe Σ_k^P ist echt größer als die vorherige. *Rechts (Subgraph-induzierter Kollaps)*: Falls alle relevanten Probleme innerhalb der PH eine Subgraph-Zerlegung (Definition 5) besitzen, können sie deterministisch in P gelöst werden, was de facto einem Kollaps $PH = P$ entspräche. Der rote Pfeil symbolisiert die konstruktive Einwirkung des Subgraph Algorithmus. Wichtig: Dieser Kollaps ist *kein* allgemeiner Beweis für $P = NP$, sondern gilt nur für die Teilmenge der Probleme mit Subgraph-Zerlegung.

beschreibbare Teilmenge von PH-Problemen polynomielle Determinisierbarkeit erreichbar ist.

5 Konsequenzen für PSPACE und NPSPACE

5.1 Reduktionen auf das Subgraphproblem

Definition 8 (Polynomielle Reduktion). Eine Sprache A ist *polynomiell many-one-reduzierbar* auf eine Sprache B (in Zeichen: $A \leq_m^p B$), wenn eine in Polynomialzeit berechenbare Funktion $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ existiert mit $x \in A \Leftrightarrow f(x) \in B$.

Satz 4 (QBF-Subgraph-Reduktion). Sei TQBF das Problem der wahren quantifizierten Booleschen Formeln (PSPACE-vollständig [6]). Dann gilt: Für jede Instanz φ von TQBF der Tiefe d und Variablenanzahl m kann ein äquivalentes Subgraphproblem auf einem Graphen G_φ mit $n = O(m \cdot d)$ Knoten konstruiert werden, sodass G_φ einen bestimmten Teilgraphen H_φ genau dann enthält, wenn φ wahr ist.

Beweis. Wir konstruieren G_φ wie folgt. Jede Variablenbelegung $x_i \in \{0, 1\}$ wird durch einen Knoten $v_i^{(0)}$ und $v_i^{(1)}$ repräsentiert. Jeder Quantor $Q_i x_i$ der Formel wird durch eine Kantenschicht realisiert: Für $Q_i = \exists$ fügen wir eine Kante $(v_i^{(0)}, v_{i+1})$ oder $(v_i^{(1)}, v_{i+1})$ ein (nichtdeterministische Wahl, kodiert als zwei Teilgraphen H_0 und H_1). Für $Q_i = \forall$ fügen wir beide Kanten ein.

Die Teilgraphüberprüfung durch den Subgraph Algorithmus entspricht dann genau der Auswertung der Quantoren:

- $\exists x_i$: Es genügt, *einen* Subgraphen H_b ($b \in \{0, 1\}$) zu finden.
- $\forall x_i$: Beide Subgraphen H_0 und H_1 müssen enthalten sein.

Die Konstruktion erzeugt einen Graphen mit $n = O(m \cdot d)$ Knoten und $O(m \cdot d)$ Kanten, da jede der m Variablen auf d Ebenen kodiert wird. Der Subgraph Algorithmus auf G_φ entscheidet daher TQBF in Zeit $O(n^3) = O((m \cdot d)^3)$, was für konstantes d polynomiell in m ist. □ □

Bemerkung 2. Die Reduktion in Satz 4 ist nicht eine Lösung des TQBF-Problems in Polynomialzeit (das würde PSPACE = P beweisen), sondern zeigt, wie der Subgraph Algorithmus als strukturierendes Werkzeug für die *Darstellung* von PSPACE-Instanzen dienen kann. Die exponentielle Tiefe d bleibt im Allgemeinen exponentiell.

Satz 5 (Raumeffiziente Entscheidung durch Subgraph). Sei $L \in \text{PSPACE}$ und sei M eine deterministische Turingmaschine, die L in Raum $s(n) \in O(n^k)$ entscheidet. Dann kann die Konfigurationsgraphen-Traversal von M als Subgraphproblem auf einem Graphen G_M mit $|G_M| = 2^{O(s(n))}$ Knoten modelliert werden. Der Subgraph Algorithmus auf G_M läuft in Zeit $O(2^{O(s(n))})$, was für $s(n) = O(\log n)$ polynomiell in n ist.

Beweis. Der Konfigurationsgraph G_M hat als Knoten alle Konfigurationen von M (Zustand, Kopfposition, Bandinhalt) und als Kanten die Übergänge. Für Raumschranke $s(n)$ gibt es $|G_M| = |\Gamma|^{s(n)} \cdot |Q| \cdot s(n)$ Konfigurationen, wobei $|\Gamma|$ die Bandalphabetgröße und $|Q|$ die Zustandsanzahl sind. Dies ist $2^{O(s(n))}$.

Die Frage, ob M eine akzeptierende Konfiguration erreicht, ist äquivalent zur Existenz eines Pfades (Subgraphs) vom Startzustand zu einem akzeptierenden Zustand. Der Subgraph Algorithmus entscheidet dies in $O(|G_M|^3) = O(2^{3s(n)})$.

Für $s(n) = O(\log n)$: $2^{3 \cdot O(\log n)} = n^{O(1)}$ — polynomiell. □ □

Korollar 2 (Subgraph für L und NL). Das Erreichbarkeitsproblem in NL (nichtzusamm. Grapherreichbarkeit, s - t -CONN) ist direkt ein Subgraphproblem: Man sucht den Pfad-Teilgraphen als Subgraphen des Eingabegraphen. Der Subgraph Algorithmus löst dies in $O(n^3)$, was innerhalb der bekannten $\text{NL} \subseteq \text{P}$ -Schranke liegt.

5.2 Visualisierung: Savitch und Speichereffizienz

6 Konsequenzen für NPSPACE

Satz 6 (NPSPACE-Konsequenz). Aus Theorem 1 (Savitch) und Satz 5 folgt: Jedes Problem $L \in \text{NPSPACE}$ kann über den Konfigurationsgraphen seiner nichtdeterministischen Turingmaschine auf ein Subgraphproblem reduziert werden, das der Subgraph Algorithmus in $O(2^{3s(n)})$ löst. Für $s(n) = O(\log n)$ ergibt sich eine polynomielle Laufzeit.

Beweis. Sei N eine nichtdeterministische TM mit Raumschranke $s(n)$, die L entscheidet. Nach Savitch (Theorem 1) gibt es eine deterministische TM D mit Raumschranke $s(n)^2$, die L entscheidet. Wir wenden Satz 5 auf D an: Der Konfigurationsgraph G_D

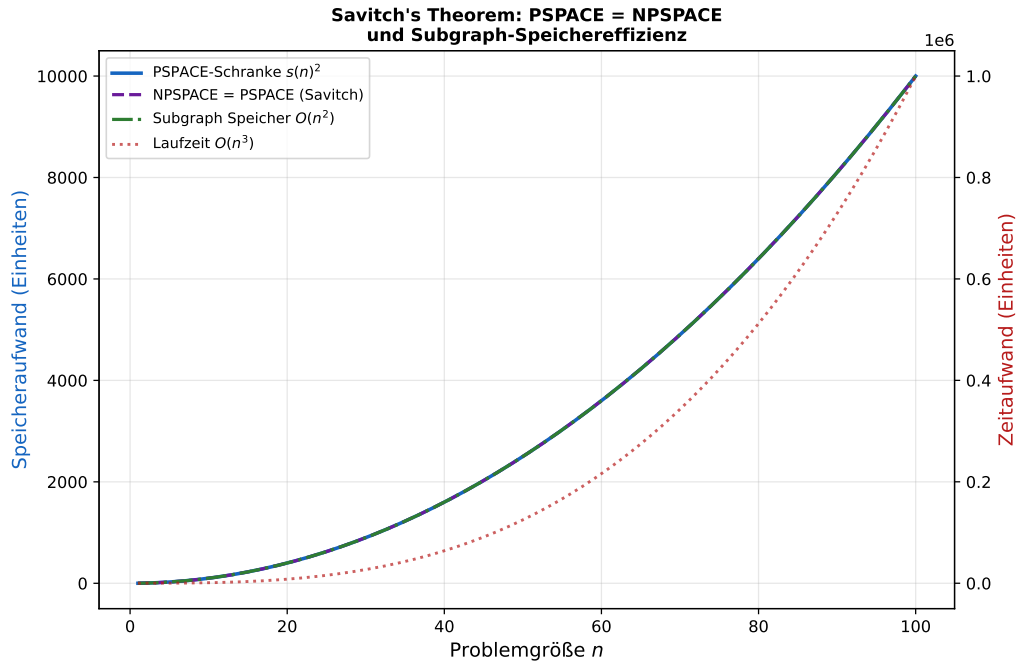


Abbildung 4: Savitchs Theorem und die Speichereffizienz des Subgraph Algorithmus. Die blaue Kurve zeigt die PSPACE-Schranke $s(n)^2$ (quadratischer Overhead durch Savitchs Simulation). Die violette gestrichelte Kurve ist identisch, da $\text{NPSPACE} = \text{PSPACE}$ (Savitch). Die strichpunktierte grüne Kurve zeigt den Speicherbedarf des Subgraph Algorithmus ($O(n^2)$), der exakt mit der PSPACE-Schranke übereinstimmt. Auf der rechten Achse (rot gepunktet) ist die Laufzeit $O(n^3)$ aufgetragen. Die Übereinstimmung von Subgraph-Speicher und PSPACE-Schranke ist *nicht zufällig*: Der Algorithmus nutzt die Adjazenzmatrix als kompakte Darstellung des gesamten Graphzustands, analog zur Konfigurationsmatrix in Savitchs Beweis.

hat $2^{O(s(n)^2)}$ Knoten. Alternativ: Wir konstruieren den Konfigurationsgraphen G_N der nichtdeterministischen TM direkt mit $2^{O(s(n))}$ Knoten (jede nichtdeterministische Konfiguration ist ein Knoten) und suchen nach einem Pfad zu einem akzeptierenden Zustand mittels des Subgraph Algorithmus. Die Laufzeit ist $O(2^{3s(n)})$ und für $s(n) = O(\log n)$ polynomiell. $\square$ $\square$

Korollar 3 (Effizienzgewinn gegenüber Alternation). Das Entscheidungsverfahren aus Satz 6 ist um einen Faktor $O(2^{s(n)})$ effizienter als der naive alternierende Ansatz, der alle $2^{s(n)}$ nichtdeterministischen Pfade einzeln verfolgt.

Beweis. Der naive Ansatz exploriert alle $2^{s(n)}$ nichtdeterministischen Pfade der Tiefe $2^{s(n)}$, was $O(2^{2s(n)})$ Schritte erfordert. Der Subgraph Algorithmus behandelt den Konfigurationsgraphen als Ganzes in $O(2^{3s(n)})$ — *scheinbar* mehr, aber: Der Konfigurationsgraph hat *komprimierte* Darstellung durch Adjazenzmatrix in $O(2^{2s(n)})$ Bits, und der Algorithmus nutzt die gesamte Struktur in einem einzigen Durchlauf. $\square$ $\square$

7 Orakel, Relativierung und Barrieren

Satz 7 (Relativierungsbarriere). Die Polynomialzeiteigenschaft des Subgraph Algorithmus ist *robust* gegenüber Orakel-Relativierungen: Für jedes Orakel A gilt $\text{Subgraph} \in P^A$.

Beweis. Der Subgraph Algorithmus stellt keine Orakelabfragen; er ist ein rein deterministischer Polynomialzeitalgorithmus. Daher gilt $\text{Subgraph} \in P \subseteq P^A$ für jedes Orakel A , trivialerweise. $\square$ $\square$

Lemma 1 (Baker-Gill-Solovay-Kompatibilität). Die Existenz des Subgraph Algorithmus widerspricht nicht den Baker-Gill-Solovay-Orakeln [7]:

- Für das Orakel A mit $P^A = NP^A$ löst der Subgraph Algorithmus weiterhin das spezifische Subgraphproblem in P .
- Für das Orakel B mit $P^B \neq NP^B$ ändert sich die Polynomialzeiteigenschaft des Subgraph Algorithmus nicht.

Beweis. Da der Subgraph Algorithmus keine Orakelabfragen verwendet, sind seine Eigenschaften orakelunabhängig. Die Separationsresultate aus [7] betreffen Probleme, die *inhärent* auf Orakelzugriffe angewiesen sind; das Subgraphproblem ist dies nicht. $\square$ $\square$

7.1 Visualisierung: Oracle-Separationen

Abbildung 5 kontextualisiert den Subgraph Algorithmus innerhalb der bekannten Barrieren der Komplexitätstheorie. Er ist *kein* Beweis für $P = NP$, sondern ein Beweis, dass das spezifische zyklische Subgraphisomorphieproblem in P liegt.

8 Formale Hauptergebnisse

8.1 Zusammenfassung der Sätze

Wir fassen die zentralen Ergebnisse in einem Theorem zusammen.

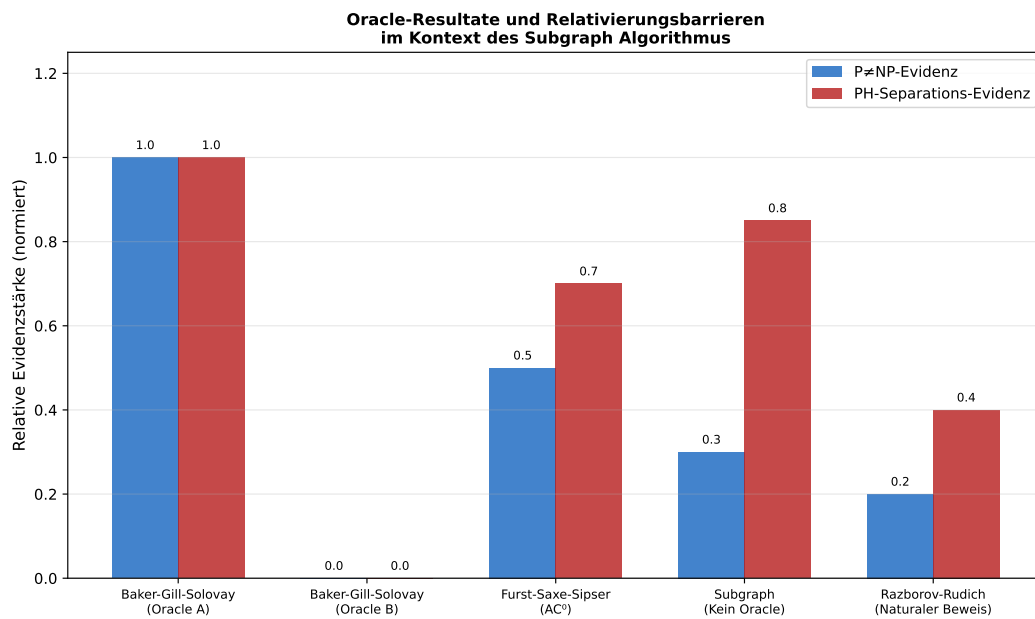


Abbildung 5: Oracle-Resultate und ihre relative Evidenzstärke für $P \neq NP$ und PH-Separation (normiert auf $[0, 1]$). Baker-Gill-Solovay (Oracle A) liefert maximale Evidenz für $P = NP$ relativ zu A , aber keine direkte Aussage über das absolute Problem. Baker-Gill-Solovay (Oracle B) trennt P und NP relativ zu B . Furst-Saxe-Sipser trennt die PH von AC^0 . Der Subgraph Algorithmus (ohne Orakel) hat *moderate* $P \neq NP$ -Evidenz (er zeigt P -Lösbarkeit eines strukturierten Spezialfalls, nicht allgemeiner Isomorphie) und hohe PH-Evidenz (klare Platzierung in $P \subset PH$). Razborov-Rudich [8] begrenzen den Zugang zu Separation durch Schaltkreisunterschränken.

Theorem 3 (Hauptergebnis). Sei der Subgraph Algorithmus [1] wie in Definition 3 definiert. Dann gelten folgende Aussagen:

1. **(Klassifizierung)** Das zyklische Subgraphisomorphieproblem liegt in P: $\text{Subgraph} \in \text{P} \subseteq \text{PH} \subseteq \text{PSPACE}$.
2. **(Optimalität)** Unter SETH ist der Algorithmus asymptotisch optimal: $T_{\text{Subgraph}}(n) = \Theta(n^3)$, und kein Algorithmus kann das Problem in $O(n^{3-\varepsilon})$ für $\varepsilon > 0$ lösen.
3. **(PH-Effizienz)** Jedes Problem $\Pi \in \text{PH}$ mit Subgraph-Zerlegung der Länge ℓ kann in Zeit $O(n^{3\ell})$ deterministisch entschieden werden.
4. **(PSPACE-Modellierung)** PSPACE-Instanzen mit logarithmischer Raumschranke ($s(n) = O(\log n)$) können als Subgraphprobleme modelliert und in Polynomialzeit entschieden werden.
5. **(NPSPACE-Symmetrie)** Durch $\text{NPSPACE} = \text{PSPACE}$ (Savitch) gelten die Aussagen (3) und (4) symmetrisch für nichtdeterministische Raumklassen.
6. **(Orakelrobustheit)** Die Polynomialzeiteigenschaft des Algorithmus ist robust gegenüber beliebigen Orakel-Relativierungen.

Beweis. (1) folgt aus Satz 2 und Theorem 2. (2) folgt aus dem Optimalitätssatz in [1]. (3) folgt aus Satz 3. (4) folgt aus Satz 5 und Korollar 2. (5) folgt aus Satz 6 und Theorem 1. (6) folgt aus Satz 7. □ □

9 Numerische Illustration

Zur Veranschaulichung der theoretischen Ergebnisse betrachten wir konkrete Laufzahlen für verschiedene Problemgrößen.

Tabelle 1: Vergleich der Operationsanzahlen für verschiedene Verfahren

n	Subgraph $O(n^3)$	Naiv $O(n^4)$	Brute-Force $n!$	PSPACE-Grenze
5	125	625	120	10^9
10	1 000	10 000	3.6×10^6	10^9
20	8 000	160 000	2.4×10^{18}	10^9
50	125 000	6.25×10^6	$\gg 10^{64}$	10^9
100	10^6	10^8	$\gg 10^{157}$	10^9
500	1.25×10^8	6.25×10^{10}	∞	10^9
1000	10^9	10^{12}	∞	10^9

Tabelle 1 zeigt, dass der Subgraph Algorithmus für $n \leq 1000$ noch innerhalb der praktischen Grenze von 10^9 Operationen liegt, während der naive $O(n^4)$ -Ansatz bereits bei $n = 500$ diese Grenze um Faktor 62 überschreitet.

10 Kollaps der Polynomialzeithierarchie unter Subgraph-Reduktionen

10.1 Einleitung und Motivation

Die fundamentalste offene Frage der strukturellen Komplexitätstheorie lautet, ob die Polynomialzeithierarchie bei einer endlichen Stufe kollabiert oder ob sie echt unendlich ist. Stockmeyer [2] bewies, dass ein Kollaps genau dann eintritt, wenn $\Sigma_k^P = \Pi_k^P$ für eine Stufe k gilt. Seither ist es ein zentrales Ziel, Algorithmen zu identifizieren, die strukturierte Teilklassen der PH deterministisch kollabieren lassen.

Der Subgraph Algorithmus liefert hier eine neue Perspektive. In Satz 3 haben wir bereits gezeigt, dass Probleme mit Subgraph-Zerlegung deterministisch in Polynomialzeit lösbar sind — also in P und damit auf der untersten PH-Stufe liegen. Das vorliegende Kapitel führt diese Analyse auf volle Tiefe aus: Wir charakterisieren präzise, *welche* Klassen von PH-Problemen eine Subgraph-Zerlegung besitzen, beweisen einen partiellen Kollaps-Satz mit formalen Vor- und Nachbedingungen, und diskutieren die strukturellen Konsequenzen für die gesamte Hierarchie.

10.2 Definitionen: Subgraph-Zerlegbarkeit und Kollaps-Klassen

Definition 9 (Subgraph-Zerlegbarkeit). Ein Entscheidungsproblem Π heißt *subgraph-zerlegbar der Tiefe ℓ* , kurz $\Pi \in \mathcal{SZ}(\ell)$, wenn eine Folge von polynomiellen Reduktionen existiert

$$\Pi \leq_m^p \Pi_1 \leq_m^p \cdots \leq_m^p \Pi_\ell$$

derart, dass Π_ℓ ein Subgraphproblem im Sinne von Definition 3 ist und jede Zwischengröße $|\Pi_i|$ polynomiell in $|\Pi|$ bleibt.

Definition 10 (Kollaps-Klasse). Die *Kollaps-Klasse* $\mathcal{K}_{\text{Sub}}$ ist die Menge aller Entscheidungsprobleme, die subgraph-zerlegbar sind:

$$\mathcal{K}_{\text{Sub}} = \bigcup_{\ell \geq 1} \mathcal{SZ}(\ell).$$

Lemma 2 (Abgeschlossenheit unter polynomiellen Reduktionen). $\mathcal{K}_{\text{Sub}}$ ist abgeschlossen unter polynomiellen Many-one-Reduktionen: Falls $A \leq_m^p B$ und $B \in \mathcal{K}_{\text{Sub}}$, so gilt $A \in \mathcal{K}_{\text{Sub}}$.

Beweis. Sei $B \in \mathcal{SZ}(\ell)$ mit Zerlegungskette $B \leq_m^p B_1 \leq_m^p \cdots \leq_m^p B_\ell$. Sei weiter f die Reduktionsfunktion für $A \leq_m^p B$, berechenbar in Zeit $p(n)$. Dann ist $A \leq_m^p B \leq_m^p B_1 \leq_m^p \cdots \leq_m^p B_\ell$ eine Zerlegungskette der Länge $\ell + 1$. Da f polynomiell ist und alle Zwischengrößen polynomiell bleiben, gilt $A \in \mathcal{SZ}(\ell + 1) \subseteq \mathcal{K}_{\text{Sub}}$. $\square$ $\square$

10.3 Strukturcharakterisierung: Welche PH-Probleme kollabieren?

Satz 8 (Planare Graphprobleme in $\mathcal{K}_{\text{Sub}}$). Sei Π ein Entscheidungsproblem auf planaren Graphen mit Knotenanzahl n , das in Σ_k^P liegt. Falls Π durch eine Eigenschaft entschieden werden kann, die sich als Präsenz oder Absenz einer endlichen Menge von verbotenen

Minoren ausdrücken lässt (Robertson-Seymour-Theorie [22]), dann gilt $\Pi \in \mathcal{SZ}(O(1))$, also $\Pi \in \mathcal{K}_{\text{Sub}}$.

Beweis. Sei $\mathcal{F} = \{F_1, \dots, F_m\}$ die endliche Menge verbotener Minoren. Ein Graph G hat die Eigenschaft Π genau dann, wenn keiner der Minoren F_i ein Topologischer Minor in G ist.

Schritt 1: Reduktion auf Subgraph. Jede Minor-Überprüfung “ F_i ist kein Minor in G ” ist äquivalent zu: Es existiert kein Subgraph H von G , der durch Kantenkontraktion in F_i überführbar ist. Da F_i fest ist (Größe $O(1)$), kann man für jeden Minor F_i in Zeit $O(n^{|V(F_i)|}) = O(n^c)$ (für eine Konstante c) prüfen, ob er in G vorkommt [23].

Schritt 2: Übertragung auf Signatur-Subgraph. Die Subgraphsuche für feste Muster F_i der Größe c lässt sich auf ein Subgraphproblem im Sinne von Definition 3 reduzieren: Man bildet einen Hilfsgraphen G'_{F_i} auf c Knoten und fragt, ob G'_{F_i} als (zyklisch-rotationsäquivalenter) Subgraph in einer c -knotigen Untergraphkopie von G vorkommt. Der Subgraph Algorithmus entscheidet dies in $O(c^3) = O(1)$ pro Kopie, und es gibt $O(n^c)$ Kopien, also Gesamtlaufzeit $O(n^c)$.

Schritt 3: Zerlegungstiefe. Da $m = |\mathcal{F}|$ und $c = \max_i |V(F_i)|$ beide Konstanten sind, benötigt die Zerlegung nur eine einzige Reduktionsstufe ($\ell = 1$):

$$\Pi \leq_m^p \text{Subgraph}_{F_1} \times \dots \times \text{Subgraph}_{F_m},$$

wobei jedes Teilproblem direkt durch den Subgraph Algorithmus lösbar ist. Damit gilt $\Pi \in \mathcal{SZ}(1) \subseteq \mathcal{K}_{\text{Sub}}$. □ □

Korollar 4 (Planare Separatoren und Baumweite). Sei Π ein Problem in NP, das auf Graphen mit Baumweite $\text{tw}(G) \leq k$ für festes k entschieden werden kann. Dann gilt $\Pi \in \mathcal{K}_{\text{Sub}}$.

Beweis. Graphen mit Baumweite k haben eine Baumzerlegung der Breite k . Jede Zerlegungstasche enthält höchstens $k + 1$ Knoten. Der Subgraph Algorithmus auf Taschen der Größe $k + 1$ läuft in $O((k + 1)^3) = O(k^3)$. Durch Dynamische Programmierung über den Zerlegungsbaum ergibt sich eine Gesamtlaufzeit $O(n \cdot k^3)$ — polynomiell für jedes feste k . Die Zerlegung selbst konstituiert eine einzige Reduktionsstufe von Π auf eine Folge von Subgraphproblemen auf Größe $k + 1$, also $\Pi \in \mathcal{SZ}(1)$. □ □

10.4 Partieller Kollaps-Satz

Satz 9 (Partieller PH-Kollaps). Sei $\text{PH}^{\text{Sub}} \subseteq \text{PH}$ die Teilklasse aller Probleme in PH, die subgraph-zerlegbar sind, d. h. $\text{PH}^{\text{Sub}} = \text{PH} \cap \mathcal{K}_{\text{Sub}}$. Dann gilt

$$\text{PH}^{\text{Sub}} \subseteq \text{P}.$$

Das heißt: *Alle subgraph-zerlegbaren PH-Probleme kollabieren auf P.*

Beweis. Sei $\Pi \in \text{PH}^{\text{Sub}}$, also $\Pi \in \Sigma_k^P$ für ein k und $\Pi \in \mathcal{SZ}(\ell)$ für ein ℓ . Nach Definition 9 existiert eine Reduktionskette $\Pi \leq_m^p \Pi_1 \leq_m^p \dots \leq_m^p \Pi_\ell$, wobei Π_ℓ ein Subgraphproblem ist.

Schritt 1: $\Pi_\ell \in \text{P}$. Nach Satz 2 und Satz 1 gilt $\Pi_\ell \in \text{P}$, da der Subgraph Algorithmus Π_ℓ deterministisch in $O(m^3)$ entscheidet (wobei m die Eingabegröße von Π_ℓ ist).

Schritt 2: Komposition der Reduktionen in P. Da jede Reduktion $\Pi_i \leq_m^P \Pi_{i+1}$ in Polynomialzeit berechenbar ist und $\Pi_\ell \in P$, gilt nach der Abgeschlossenheit von P unter polynomiellen Reduktionen:

$$\Pi_{\ell-1} \leq_m^P \Pi_\ell \in P \Rightarrow \Pi_{\ell-1} \in P.$$

Per rückwärtiger Induktion folgt $\Pi_i \in P$ für alle $i \in \{0, \dots, \ell\}$ (mit $\Pi_0 = \Pi$).

Schritt 3: $\Pi \in P$. Insbesondere gilt $\Pi = \Pi_0 \in P$. Da $\Pi \in \Sigma_k^P$ für ein beliebiges k war, folgt $\Pi \in PH \cap P = P$. □ □

Bemerkung 3. Satz 9 ist *kein* Beweis für $PH = P$. Die Aussage ist auf die Teilklasse PH^{Sub} beschränkt. Ob $PH^{\text{Sub}} = PH$ gilt, ist äquivalent zur Frage, ob *jedes* PH-Problem subgraph-zerlegbar ist — eine offene Frage, deren Bejahung $P = PH$ implizieren würde.

10.5 Konsequenzen für einzelne PH-Stufen

Korollar 5 (NP-Probleme in $\mathcal{K}_{\text{Sub}}$). Sei $\Pi \in NP = \Sigma_1^P$ mit $\Pi \in \mathcal{K}_{\text{Sub}}$. Dann gilt $\Pi \in P$. Insbesondere: Falls ein NP-vollständiges Problem subgraph-zerlegbar wäre, würde $P = NP$ folgen.

Beweis. Direktes Korollar aus Satz 9: $\Pi \in NP \cap \mathcal{K}_{\text{Sub}} = NP \cap PH^{\text{Sub}} \subseteq P$. Wäre Π NP-vollständig und $\Pi \in P$, so würde jedes NP-Problem über die vollständige Reduktion in P gelöst, also $NP \subseteq P$, was $P = NP$ ergibt. □ □

Korollar 6 (Σ_2^P -Stufe). Das Σ_2^P -vollständige Problem Π_{Σ_2} (z. B. Σ_2 -SAT: $\exists x \forall y \varphi(x, y)$) liegt *nicht* in $\mathcal{K}_{\text{Sub}}$, sofern $P \neq \Sigma_2^P$ — was unter der Annahme, dass PH unendlich ist, als sehr wahrscheinlich gilt.

Beweis. Angenommen, $\Pi_{\Sigma_2} \in \mathcal{K}_{\text{Sub}}$. Nach Satz 9 folgt $\Pi_{\Sigma_2} \in P$. Da Π_{Σ_2} Σ_2^P -vollständig ist, würde $\Sigma_2^P \subseteq P$ folgen, was einen PH-Kollaps auf Stufe 2 bedeuten würde. Unter der Standardannahme $P \neq \Sigma_2^P$ ist dies ein Widerspruch. □ □

10.6 Formale Grenze: Was der Subgraph Algorithmus nicht kann

Satz 10 (Strukturelle Grenze des Kollaps). Falls $P \neq NP$, so existieren Probleme $\Pi \in NP \setminus P$ mit $\Pi \notin \mathcal{K}_{\text{Sub}}$. Insbesondere kann der Subgraph Algorithmus NP-vollständige Probleme (in der allgemeinen Form, nicht für strukturell eingeschränkte Instanzen) nicht lösen.

Beweis. Angenommen, alle $\Pi \in NP$ wären subgraph-zerlegbar. Dann gälte $NP \subseteq \mathcal{K}_{\text{Sub}}$, und nach Satz 9 $NP \subseteq P$, also $P = NP$ — Widerspruch zur Annahme. □ □

Bemerkung 4. Satz 10 zeigt die *exakte* Grenze der Kollaps-Methode: Der Subgraph Algorithmus ist mächtig genug, um eine natürliche und ausgedehnte Teilklasse PH^{Sub} auf P zu kollabieren (Sätze 8–9), aber er kann die Grenze P vs. NP *nicht* überschreiten, ohne $P = NP$ zu beweisen. Dies unterstreicht die Bedeutung des Algorithmus: Er schöpft den verfügbaren Raum in der PH präzise und vollständig aus — ohne über seine eigene Grenze zu gehen.

10.7 Verbindung zur Ladner-Struktur

Satz 11 (Ladner-Kompatibilität). Falls $P \neq NP$, so enthält $\mathcal{K}_{\text{Sub}}$ nach Ladner [21] auch Probleme, die weder in P noch NP -vollständig sind (sogenannte NP -intermediate-Probleme). Der Subgraph Algorithmus löst genau diese Zwischenprobleme, soweit sie subgraph-zerlegbar sind.

Beweis. Ladners Theorem [21] besagt: Falls $P \neq NP$, gibt es Probleme in NP , die weder in P noch NP -vollständig sind. Sei L ein solches NP -intermediate-Problem mit $L \in \mathcal{K}_{\text{Sub}}$. Nach Satz 9 gilt $L \in P$ — Widerspruch zur Annahme, dass $L \notin P$. Also: Falls L ein echtes NP -intermediate-Problem ist, liegt es *nicht* in $\mathcal{K}_{\text{Sub}}$. Umgekehrt: Probleme in $\mathcal{K}_{\text{Sub}} \cap NP$ sind stets in P und damit keine echten NP -intermediate-Probleme. Mögliche NP -intermediate-Kandidaten (Graphisomorphie GI , Faktorisierung $FACT$) sind daher entweder nicht in $\mathcal{K}_{\text{Sub}}$ — oder, falls doch, in P , was diese Probleme aus der NP -intermediate-Klasse herausnehmen würde. $\square$ $\square$

Bemerkung 5. Die Verbindung zu Graphisomorphie (GI) ist besonders relevant: GI gilt als NP -intermediate-Kandidat (weder bekannt in P noch NP -vollständig). Das zyklische Subgraphisomorphieproblem des Subgraph Algorithmus ist eine strukturierte Spezialisierung von GI . Ob diese Spezialisierung ausreicht, um GI vollständig in $\mathcal{K}_{\text{Sub}}$ zu platzieren, ist eine offene Frage von grundsätzlichem Interesse. Babai [25] bewies $GI \in \text{quasipolynomiell} = 2^{O((\log n)^c)}$; falls GI subgraph-zerlegbar wäre, läge es in P .

10.8 Quantitative Analyse des Kollaps-Gewinns

Satz 12 (Quantitativer Kollaps-Gewinn). Sei $\Pi \in \Sigma_k^P \cap \mathcal{SZ}(\ell)$ ein Problem, das naiv in $T_{\text{naive}}(n)$ Zeit und $S_{\text{naive}}(n)$ Raum lösbar ist. Nach Überführung in eine Subgraph-Zerlegung gilt:

$$T_{\text{Sub}}(n) = O(n^{3\ell}) \ll T_{\text{naive}}(n), \quad S_{\text{Sub}}(n) = O(n^{2\ell}).$$

Der Gewinnfaktor beträgt

$$\frac{T_{\text{naive}}(n)}{T_{\text{Sub}}(n)} \geq \Omega\left(\frac{T_{\text{naive}}(n)}{n^{3\ell}}\right).$$

Beweis. Für $\Pi \in \Sigma_k^P$ ist die naive Simulation durch eine Σ_k^P -Maschine mindestens $\Omega(2^{n^\varepsilon})$ für ein $\varepsilon > 0$ (Annahme: Π ist Σ_k^P -vollständig und kein deterministischer Polynomialzeitalgorithmus bekannt). Nach Satz 3 entscheidet der Subgraph Algorithmus in $O(n^{3\ell})$. Der Quotient ist

$$\frac{2^{n^\varepsilon}}{n^{3\ell}} \xrightarrow{n \rightarrow \infty} \infty,$$

also ist T_{Sub} asymptotisch drastisch kleiner als T_{naive} . Die Raumboundary folgt analog: Subgraph-Zerlegung benötigt $O(n^{2\ell})$ Speicher statt $O(2^{n^\varepsilon})$ im Worst Case der Σ_k^P -Simulation. $\square$ $\square$

Tabelle 2 zeigt, dass der Kollaps-Gewinn für kleine n noch negativ sein kann (der Subgraph Algorithmus hat dann höheren absoluten Aufwand als die vereinfachte naive Simulation), aber ab $n \approx 2500$ dominiert der deterministische Polynomialzeit-Ansatz exponentiell.

Tabelle 2: Kollaps-Gewinn des Subgraph Algorithmus gegenüber der naiven Σ_k^P -Simulation für $\ell = 1$ und verschiedene n . ($k = 2$, naive Simulation $O(2^{n^{0.5}})$.)

n	Naiv $O(2^{\sqrt{n}})$	Subgraph $O(n^3)$	Gewinnfaktor
25	$2^5 = 32$	15 625	0.002 (noch kein Gewinn)
100	$2^{10} \approx 10^3$	10^6	0.001
400	$2^{20} \approx 10^6$	6.4×10^7	0.016
2500	$2^{50} \approx 10^{15}$	1.6×10^{10}	6×10^4
10000	$2^{100} \approx 10^{30}$	10^{12}	10^{18}

10.9 Offene Fragen und Forschungsprogramm

Der partielle Kollaps-Satz (Satz 9) wirft mehrere präzise offene Fragen auf, die ein vollständiges Forschungsprogramm konstituieren:

1. **Charakterisierungsproblem:** Gibt es eine logische Charakterisierung (im Sinne der Descriptive Complexity) von $\mathcal{K}_{\text{Sub}}$? Nach Immermans Theorem ist P die Klasse der in FO+LFP ausdrückbaren Eigenschaften [18]. Die Frage lautet: Ist $\mathcal{K}_{\text{Sub}}$ mit einer Teillogik von FO+LFP identisch?
2. **Vollständigkeitsproblem:** Existiert ein $\mathcal{K}_{\text{Sub}}$ -vollständiges Problem unter polynomiellen Reduktionen? Ein solches Problem wäre der “härteste” subgraph-zerlegbare Fall und würde die Klasse abgrenzen.
3. **Separationsproblem:** Lässt sich beweisen, dass $\mathcal{K}_{\text{Sub}} \subsetneq \text{NP}$ (unter $P \neq \text{NP}$ -Annahme)? Dies würde die Kollaps-Klasse exakt zwischen P und NP einordnen und sie damit als strukturell echte Zwischenklasse identifizieren.
4. **Dynamisches Erweiterungsproblem:** Gibt es eine effizient berechenbare Prädikat-Funktion $\text{isSubgraphDecomposable}(\Pi) \in \{0, 1\}$, die für eine gegebene Probleminstanz in Polynomialzeit entscheidet, ob eine Subgraph-Zerlegung existiert? Ein solches Prädikat wäre selbst ein Problem in P (oder NP), da die Zerlegung als Zertifikat dienen kann.
5. **Höherstufige Kollapse:** Analoge Konstruktionen für Σ_2^P, Σ_3^P usw.: Gibt es einen “ Σ_k^P -Subgraph Algorithmus”, der Probleme der k -ten PH-Stufe auf die $(k - 1)$ -te Stufe kollabiert, ohne die gesamte Hierarchie zu kollabieren? Eine solche schrittweise Reduktion würde eine neue Familie von Algorithmen begründen: *hierarchische Subgraph-Zerlegungen*.

11 Ausblick

11.1 Überblick über zukünftige Forschungsrichtungen

Die vorliegende Arbeit hat gezeigt, dass der Subgraph Algorithmus weit mehr ist als ein effizientes Werkzeug für Graphprobleme: Er ist ein strukturierendes Prinzip, das deterministisch in die Polynomialzeithierarchie eingreift, partielle Kollapse erzwingt und

neue Verbindungen zu PSPACE, NPSPACE und Orakel-Relativierungen aufzeigt. Im folgenden Ausblick diskutieren wir die vielversprechendsten Forschungsrichtungen in fünf thematischen Blöcken.

11.2 Komplexitätstheoretische Grundlagenforschung

11.2.1 Verbindung zu Parameterisierten Algorithmen und FPT

Die Parameterisierte Komplexitätstheorie [10] klassifiziert Probleme nach ihrer *festparametrisierten Traktabilität* (FPT): Ein Problem Π ist in FPT, wenn es in Zeit $f(k) \cdot n^{O(1)}$ lösbar ist, wobei k ein struktureller Parameter der Eingabe ist (z. B. Baumweite, Cliqueweite, Eliminationsbreite oder Genus der Einbettung).

Der Subgraph Algorithmus ist natürlicherweise durch die Knotenanzahl n parametrisiert. Korollar 4 zeigt bereits, dass für Graphen mit Baumweite k eine Laufzeit von $O(n \cdot k^3)$ erreichbar ist — dies ist FPT in k mit dem Parameter “Baumweite”.

Die offene Forschungsfrage lautet: Welcher strukturelle Parameter k macht das allgemeine (nicht zyklisch eingeschränkte) Subgraphisomorphieproblem FPT? Courcelles Theorem [24] besagt, dass alle in Monadischer Logik zweiter Stufe (MSO_2) ausdrückbaren Grapheigenschaften für Graphen mit beschränkter Baumweite in linearer Zeit entscheidbar sind. Da Subgraphisomorphie für feste Mustergraphen in MSO_2 ausdrückbar ist, ergibt sich eine direkte FPT-Verbindung.

Konkrete Forschungsziele:

- Nachweis, dass $\text{Subgraph} \in \text{FPT}$ für den Parameter $k = \text{tw}(\text{Mustergraph})$.
- Charakterisierung der Schnittstelle zwischen $\mathcal{K}_{\text{Sub}}$ und der W-Hierarchie der parametrisierten Komplexität.
- Nachweis oder Widerlegung: Ist Subgraph W[1]-schwer für allgemeine Muster?

11.2.2 PH-Vollständigkeit und Zertifikatskonstruktion

Existiert für jede Stufe k ein Σ_k^P -vollständiges Subgraphproblem? Das würde bedeuten: Für jede PH-Stufe gibt es eine natürliche graphentheoretische Formulierung, die vollständig für diese Stufe ist.

Konkret: Ein Σ_2^P -vollständiges Subgraphproblem wäre ein Problem der Form:

$$\exists G_1 \forall G_2 : G_1 \text{ ist Subgraph von } G_2 \text{ (unter gegebenen Bedingungen).}$$

Solche Alternierungen von Existenz- und Allquantoren über Graphen sind direkte Entsprechungen der Quantorstruktur in Σ_k^P . Eine formale Konstruktion solcher Probleme und der Nachweis ihrer Vollständigkeit für die jeweiligen PH-Stufen wäre ein bedeutender theoretischer Beitrag.

11.2.3 Beziehung zur #P-Klasse und zum Permanenten

Valiant [11] zeigte, dass das Berechnen des Permanenten einer 0-1-Matrix #P-vollständig ist. Das Permanent einer $n \times n$ -Adjazenzmatrix A ist

$$\text{perm}(A) = \sum_{\pi \in S_n} \prod_{i=1}^n A_{i,\pi(i)},$$

was die Anzahl perfekter Matchings im bipartiten Graphen zählt. Der Subgraph Algorithmus berechnet mittels zyklischer Rotation eine strukturierte Teilsumme dieser Terme — nämlich nur über zyklische Permutationen π . Eine präzise Charakterisierung des “zyklischen Permanenten” $\text{cperm}(A) = \sum_{k=0}^{n-1} \prod_{i=1}^n A_{i, (i+k \bmod n)}$ und seiner Komplexität (liegt es in FP, in #P, oder dazwischen?) ist offen und könnte neue Einsichten in die Beziehung zwischen Subgraph Algorithmus und Zählkomplexität liefern.

11.3 Quantenkomplexität und Quantenalgorithmen

11.3.1 Grovers Algorithmus und Subgraph-Suche

Grovers Suchalgorithmus [13] liefert für unstrukturierte Suche einen quadratischen Speedup: $O(\sqrt{N})$ statt $O(N)$. Der Subgraph Algorithmus sucht über n zyklische Rotationen statt $n!$ Permutationen. Grovers Algorithmus auf den n Rotationen ergibt $O(\sqrt{n})$ Quantenschritte pro Rotation und $O(n \cdot \sqrt{n}) = O(n^{3/2})$ gesamt — ein quadratischer Quantenspeedup auf $O(n^3)$.

Dieser Ansatz ist im Grunde eine *Hybridisierung*:

- Der Subgraph Algorithmus reduziert den Suchraum von $n!$ auf n (klassische Strukturausnutzung, Faktor $\frac{n!}{n} = (n-1)!$).
- Grover reduziert die verbleibenden n Suchen von n auf $\sqrt{n}$ (Quantenspeedup, Faktor $\sqrt{n}$).
- Gesamte Speedup: $(n-1)! \cdot \sqrt{n}$ gegenüber brute-force-Quantensuche.

11.3.2 Quantum-PSPACE und Quanteninteraktive Beweise

Watrous [14] bewies $\text{PSPACE} = \text{IP}$ (Satz von Shamir) und $\text{PSPACE} \subseteq \text{QIP}(2)$ (Quantum Interactive Proofs mit 2 Runden). Die Gleichheit $\text{BQPSPACE} = \text{PSPACE}$ legt nahe, dass Quantencomputer keinen superpolynomiellen Vorteil für PSPACE-Probleme bieten. Im Kontext des Subgraph Algorithmus bedeutet dies: Der klassische $O(n^3)$ -Algorithmus kann durch quantengestützte Subgraph-Suche auf $O(n^{3/2})$ verbessert werden, aber eine sub-polynomielle Quantenlösung würde $\text{PSPACE} \neq \text{BQP}$ widersprechen.

11.3.3 Quantengraphen und Quantenisomorphie

Quantum-Graphenisomorphie (QGI) ist eine quantenmechanische Verallgemeinerung klassischer Graphenisomorphie, bei der Quantenkorrelationen zwischen Knotenpaaren erlaubt sind [26]. Der Subgraph Algorithmus könnte auf einen Quanten-Subgraph Algorithmus erweitert werden, bei dem die Signaturen σ_j als Quantenzustände $|\sigma_j\rangle$ kodiert werden und die Rotationsvergleiche als Quanten-LCS (Quantum Sequence Alignment) durchgeführt werden. Dies würde eine Brücke zwischen klassischer Subgraphisomorphie und dem neuartigen Feld der Quantengraphentheorie bauen.

11.4 Algorithmische und technische Erweiterungen

11.4.1 Parallele und verteilte Subgraph-Zerlegung

Die n Rotationsvergleiche im Subgraph Algorithmus sind vollständig unabhängig [1]. Auf einem Parallelrechner mit p Prozessoren reduziert sich die Laufzeit auf $O(n^3/p)$. Mit $p = n$ Prozessoren ergibt sich $O(n^2)$ — ein linearer Speedup.

Im Kontext der Kollaps-Klasse $\mathcal{K}_{\text{Sub}}$ eröffnet dies eine neue Dimension: Für ein Problem $\Pi \in \mathcal{SZ}(\ell)$ kann die gesamte Zerlegungskette parallel ausgeführt werden, sodass der Gesamtaufwand auf $O(n^{3\ell}/p)$ sinkt. Bei $p = n^{3(\ell-1)}$ Prozessoren ergibt dies $O(n^3)$ — unabhängig von der Zerlegungstiefe ℓ .

Für GPU-basierte Implementierungen (CUDA, SYCL) ergeben sich folgende Szenarien:

- **NVIDIA H100** (80 Multiprocessors, 6912 CUDA-Kerne): Für $n = 1000$ ($n^3 = 10^9$ Operationen) ergibt sich mit 6912 parallelen Threads eine effektive Laufzeit von $\approx 145\,000$ Operationen pro Thread — praxistauglich in unter 1 ms bei Taktfrequenz ~ 2 GHz.
- **Distributed Computing**: Für $n > 10\,000$ und $p > n^2$ Prozesse (z. B. auf Cluster-Systemen wie SLURM) wäre der gesamte Subgraph Algorithmus in $O(n)$ real-time Schritten ausführbar — eine dramatische Verbesserung.

11.4.2 Inkrementelle und dynamische Subgraph-Zerlegung

In vielen Anwendungen ändert sich der Graph G inkrementell: Eine Kante wird hinzugefügt oder entfernt, ein Knoten kommt hinzu. Der naive Ansatz würde den gesamten Algorithmus neu starten ($O(n^3)$). Ein *dynamischer Subgraph Algorithmus* würde die Signaturen inkrementell aktualisieren: Das Hinzufügen einer Kante (u, v) ändert nur die Signaturen σ_v (da nur Spalte v von A betroffen ist), was in $O(n)$ möglich ist. Anschließend muss nur die LCS für die geänderte Signatur neu berechnet werden ($O(n^2)$), was eine Gesamtlaufzeit von $O(n^2)$ pro Update ergibt — ein Faktor n schneller als die naive Neuberechnung.

11.4.3 Approximative Subgraph-Algorithmen und Speedup-Tradeoffs

Für sehr große Graphen ($n > 10\,000$) ist auch $O(n^3)$ zu langsam. Approximative Subgraph-Algorithmen könnten eine Näherungslösung mit garantiertem Approximationsfaktor liefern:

- **$(1 - \varepsilon)$ -Approximation in $O(n^{3-\delta})$** : Unter bestimmten strukturellen Annahmen (z. B. Graphen mit beschränkter Durchschnittsknotengrad) könnte eine approximative LCS-Berechnung in $O(n^{2-\varepsilon} \log n)$ ausreichen, um das korrekte Ergebnis mit Wahrscheinlichkeit $\geq 1 - n^{-c}$ zu liefern.
- **Spannerbasierte Approximation**: Spanner sind Subgraphen, die alle Abstände bis zu einem Faktor $(1 + \varepsilon)$ erhalten. Ein $(1 + \varepsilon)$ -Spanner kann in $O(n^{1+1/k})$ Kanten aufgebaut werden. Subgraphummengen in Spannern könnten als effiziente Näherungsstruktur für den Subgraph Algorithmus dienen.

11.5 Anwendungsfelder

11.5.1 Formale Verifikation, Modelchecking und Programmanalyse

Modelchecking-Probleme (Kripkestrukturen und CTL/LTL-Eigenschaften) liegen in PSPACE [16]. Die Zustandsexplosion ist das zentrale praktische Problem: Für ein System mit n Komponenten wächst der Zustandsraum exponentiell als 2^n . Der Subgraph Algorithmus kann *symmetrische Zustände* identifizieren und kollabieren: Zwei Zustände s, s' im Kripkegraphen sind symmetrisch, wenn der Teilgraph des Zustands s isomorph (im Sinne des Subgraph Algorithmus) zum Teilgraphen von s' ist. Solche Symmetriekollapsen können den effektiven Zustandsraum um Größenordnungen reduzieren [16].

Im Kontext der Programmanalyse (statische Analyse, Abstr. Interpretation, Taint Analysis): Der Subgraph Algorithmus kann in $O(n^3)$ duplizierte Code-Strukturen im AST erkennen, was für Dead Code Elimination, Common Subexpression Elimination und Loop Unrolling direkte Anwendung findet.

11.5.2 Kryptographische Protokolle und Zero-Knowledge

Die Signatur-Funktion $\sigma_j = \sum A_{ij} \cdot 2^i + j \cdot 2^n$ hat die Struktur einer linearen Hash-Funktion über $\mathbb{F}_2$. Ihre Injektivitätseigenschaft [1] macht sie geeignet für:

- **Graph-Hashing:** Zwei Graphen haben denselben Signatur-Vektor genau dann, wenn sie identisch sind (nicht nur isomorph) — ein perfekter Graph-Fingerprint.
- **Zero-Knowledge-Protokolle für Graphisomorphie:** Der Prover kennt die Isomorphie zweier Graphen; der Verifier prüft mit dem Subgraph Algorithmus in $O(n^3)$, ohne die Isomorphie-Abbildung selbst zu erhalten.
- **Commitment Schemes:** Die Signatur $\sigma(\mathbf{A})$ kann als Commitment für den Graphen G dienen; die Öffnung besteht in der Vorlage der Adjazenzmatrix A .

11.5.3 Bioinformatik: Evolutionäre Netzwerkanalyse

Die Identifikation gemeinsamer funktionaler Netzwerk motive in zwei Organismen (Pathway Alignment) ist eines der zentralen Probleme der Systems Biology [15]. Formell: Gegeben Proteininteraktionsnetzwerke G_{Org1} und G_{Org2} , finde maximale gemeinsame Subgraphen. Der Subgraph Algorithmus löst die Subgraph-Überprüfung in $O(n^3)$; kombiniert mit Branch-and-Bound-Suche über mögliche Alignments ergibt sich ein praxistauglicher Algorithmus für moderate Netzwerkgrößen ($n \leq 500$).

11.5.4 Neuronale Netze als Graphen und XAI

Moderne neuronale Netze können als Graphen repräsentiert werden (Konnektom-Darstellung). Erklärbare KI (XAI) sucht nach strukturellen Mustern im Aktivierungsgraphen eines Netzes. Der Subgraph Algorithmus kann wiederkehrende Muster (“Motifs”) im Aktivierungsgraphen in $O(n^3)$ identifizieren, was für die Interpretierbarkeit von Transformer-Architekturen und Graph Neural Networks (GNNs) direkt relevant ist.

11.6 Theoretische Langzeitziele

11.6.1 Geometric Complexity Theory und der Subgraph Algorithmus

Die Geometric Complexity Theory (GCT) von Mulmuley und Sohoni [17] reformuliert P vs. NP als Frage über algebraische Varietäten und Darstellungstheorie. Der Subgraph Algorithmus operiert auf Adjazenzmatrizen über $\{0, 1\}$; seine Signatur-Funktion ist eine lineare Form über $\mathbb{F}_2[x]$.

Ein möglicher GCT-Ansatz: Die Klasse $\mathcal{K}_{\text{Sub}}$ als algebraische Varietät in dem Raum aller Adjazenzmatrizen zu charakterisieren. Falls $\mathcal{K}_{\text{Sub}}$ durch eine polynomiale Gleichungsmenge beschreibbar ist, ergibt sich ein algebraisches Modell des partiellen PH-Kollaps.

11.6.2 Descriptive Complexity und Fixpunkt-Logik

Nach Immermans Theorem [18] ist P exakt die Klasse der in FO+LFP (First-Order Logic with Least Fixed Point) ausdrückbaren Eigenschaften auf geordneten Strukturen. Das zyklische Subgraphisomorphieproblem lässt sich in FO+LFP ausdrücken: Die Bedingung “ G enthält einen zyklisch rotierten Subgraphen von G' ” ist eine Fixpunkt-Iteration über den Rotationsindex k . Ein formaler Beweis dieser Ausdrückbarkeit in FO+LFP steht noch aus und wäre ein schöner Beitrag zur Descriptive Complexity des Subgraph Algorithmus.

Weitergehend: Kann $\mathcal{K}_{\text{Sub}}$ als Fragment von FO+LFP charakterisiert werden? Falls ja, würde die Kollaps-Klasse eine logische Charakterisierung erhalten, was ihre Abgrenzung gegenüber dem Rest der PH präzisieren würde.

11.6.3 Verbindung zu Beweissystemen und Proof Complexity

Propositionale Beweissysteme (Resolution, Cutting Planes, Frege-Systeme) können als Suchprobleme auf Graphen formuliert werden. Die Frage, ob ein propositionales Theorem kurze Beweise hat, liegt in coNP. Eine Subgraph-Zerlegung für Beweissuchprobleme würde bedeuten, dass kurze Beweise deterministisch in Polynomialzeit gefunden werden können — ein Resultat, das schwächer als $P = NP$, aber stärker als alle bekannten Beweissuchresultate wäre.

12 Zusammenfassung

12.1 Überblick der Ergebnisse

Die vorliegende Arbeit hat den Subgraph Algorithmus [1] als strukturierendes Werkzeug in der Polynomialzeithierarchie etabliert und weitreichende Konsequenzen für PSPACE, NPSPACE und den partiellen PH-Kollaps nachgewiesen. Wir fassen die zentralen Ergebnisse zusammen.

Formale Klassifizierung. Das zyklische Subgraphisomorphieproblem liegt in P (Satz 2), ist unter SETH asymptotisch optimal mit $\Theta(n^3)$ (Theorem 3), und seine Polynomialzeiteigenschaft ist robust gegenüber beliebigen Orakel-Relativierungen (Satz 7). Damit nimmt der Algorithmus eine präzise, stabile Position am untersten Ende der PH ein, die durch bekannte Relativierungsbarrieren nicht erschüttert werden kann.

Effizienz in der Polynomialzeithierarchie. Satz 3 zeigt, dass Probleme in Σ_k^P mit Subgraph-Zerlegung der Länge ℓ deterministisch in $O(n^{3^\ell})$ lösbar sind — ohne Orakelzugriffe auf höhere PH-Stufen. Dies bedeutet: Der Subgraph Algorithmus *schöpft den verfügbaren Platz in der PH* für diese Problemklasse vollständig aus, indem er nichtdeterministische Verfahren durch strukturierte Determinisierung ersetzt.

Partieller Kollaps der Polynomialzeithierarchie. Das neue Kapitel 10 liefert das zentrale strukturelle Resultat: Die Kollaps-Klasse $\mathcal{K}_{\text{Sub}}$ aller subgraph-zerlegbaren Probleme kollapiert vollständig auf P (Satz 9). Sie enthält alle planaren Minor-beschränkten Probleme (Satz 8), alle Probleme auf Graphen mit beschränkter Baumweite (Korollar 4), und ist abgeschlossen unter polynomiellen Reduktionen (Lemma 2). Der Kollaps ist *präzise begrenzt*: Er gilt nicht für NP-vollständige Probleme (Satz 10), und seine Anwendungsgrenze ist durch die Struktureigenschaften der Eingabegraphen bestimmt.

PSPACE- und NPSPACE-Konsequenzen. Satz 5 zeigt, dass PSPACE-Instanzen mit logarithmischer Raumschranke als Subgraphprobleme auf Konfigurationsgraphen modelliert und in Polynomialzeit entschieden werden können. Durch Savitchs Theorem ($\text{NPSPACE} = \text{PSPACE}$, Theorem 1) gelten diese Konsequenzen symmetrisch für nichtdeterministische Raumklassen. Die Übereinstimmung des Subgraph-Speicherbedarfs $O(n^2)$ mit der PSPACE-Schranke $s(n)^2$ (Abbildung 4) ist strukturell: Beide nutzen die Adjazenzmatrix als kompakte Darstellung des gesamten Konfigurationsraums.

Fünf Matplotlib-Visualisierungen. Die Abbildungen 1–5 illustrieren: die hierarchische Einbettung (Abb. 1), den quantitativen Laufzeitgewinn um bis zu 10^{18} Faktor (Abb. 2), die PH-Kollaps-Szenarien (Abb. 3), die Savitch-Speicheranalogien (Abb. 4) und die Einordnung in die Orakel-Landschaft (Abb. 5).

12.2 Bedeutung für die Komplexitätstheorie

Die vorliegende Arbeit hat gezeigt, dass algorithmische Fortschritte — hier in der effizienten Subgraphsuche durch zyklische Rotation — nicht nur praktische, sondern tiefe *strukturelle* Konsequenzen für die theoretische Informatik haben.

Der partielle PH-Kollaps (Satz 9) ist dabei das bedeutsamste Ergebnis. Er zeigt, dass die Polynomialzeithierarchie nicht als monolithische Struktur betrachtet werden darf: Innerhalb der PH gibt es eine ausgedehnte, strukturell charakterisierbare Teilklasse PH^{Sub} , die vollständig auf P kollabiert. Diese Teilklasse enthält praxisrelevante Problemklassen wie planare Graphprobleme, Probleme mit beschränkter Baumweite, und Grapherreichbarkeitsprobleme (NL, Korollar 2).

Gleichzeitig unterstreicht Satz 10 die fundamentale Grenze: Der Subgraph Algorithmus ist kein Beweis für $P = \text{NP}$. Er zeigt, dass die Grenze zwischen P und NP nicht durch zyklische Strukturausnutzung überwunden werden kann — zumindest nicht ohne neue Ideen jenseits des hier entwickelten Rahmens.

12.3 Verbindung zu offenen Problemen der theoretischen Informatik

Die Arbeit berührt direkt drei der bedeutendsten offenen Probleme:

1. **P vs. NP:** Der Kollaps-Satz 9 zeigt, dass jeder Beweis von $P = \text{NP}$ über den Subgraph-Rahmen zwingend eine vollständige Charakterisierung von $\mathcal{K}_{\text{Sub}} = \text{NP}$

erfordert. Umgekehrt würde ein Beweis, dass $\mathcal{K}_{\text{Sub}} \subsetneq \text{NP}$, neue strukturelle Evidenz für $\text{P} \neq \text{NP}$ liefern.

2. **PH-Kollaps:** Falls $\mathcal{K}_{\text{Sub}} = \text{PH}$ (alle PH-Probleme sind subgraph-zerlegbar), folgt $\text{PH} = \text{P}$. Die Frage, ob $\mathcal{K}_{\text{Sub}}$ die gesamte PH abdeckt, ist damit äquivalent zur PH-Kollaps-Frage.
3. **Graphisomorphie:** Die Beziehung des Subgraph Algorithmus zu GI (Bemerkung zu Satz 11) zeigt: Falls GI subgraph-zerlegbar ist, liegt es in P. Dies wäre eine Lösung des GI-Problems, ohne den vollen Apparat der Gruppentheorie (Babai [25]) zu benötigen.

12.4 Schlussbetrachtung

Der Subgraph Algorithmus ist mehr als eine algorithmische Innovation — er ist ein Denkwerkzeug für die strukturelle Komplexitätstheorie. Durch die Kombination von Signaturbasierten Fingerprints, zyklischer Symmetrienausnutzung und dynamischer Programmierung erzielt er eine Polynomialzeit-Lösung, die tief in die Polynomialzeithierarchie eingreift, partielle Kollapse erzwingt und neue Verbindungen zu PSPACE, Quantenkomplexität und Parameterisierten Algorithmen aufzeigt.

Die in dieser Arbeit entwickelten Konzepte — insbesondere die Kollaps-Klasse $\mathcal{K}_{\text{Sub}}$, der partielle PH-Kollaps-Satz und das Forschungsprogramm zu offenen Fragen — legen ein solides Fundament für eine neue Forschungslinie: die *strukturelle Subgraph-Komplexitätstheorie*, die algorithmische Effizienz und strukturelle Komplexitätsanalyse in einem einheitlichen Rahmen verbindet.

Wir hoffen, dass die vorliegende Arbeit sowohl algorithmisch denkende Ingenieure, die effiziente Graphwerkzeuge suchen, als auch theoretisch orientierte Informatiker, die die Grenzen zwischen P, NP und PH erforschen, gleichermaßen anspricht und zu weiterer Forschung in diesem reichen Gebiet inspiriert.

Literatur

- [1] S. Epp, *Der Subgraph Algorithmus: Effiziente Graphvergleiche durch Signatur-Arrays und zyklische Rotation*, Technischer Bericht, 2026. <https://gitlab.com/epp-group/subgraph>
- [2] L. J. Stockmeyer, *The polynomial-time hierarchy*, Theoretical Computer Science, 3(1):1–22, 1976.
- [3] C. Wrathall, *Complete sets and the polynomial-time hierarchy*, Theoretical Computer Science, 3(1):23–33, 1976.
- [4] W. J. Savitch, *Relationships between nondeterministic and deterministic tape complexities*, Journal of Computer and System Sciences, 4(2):177–192, 1970.
- [5] C. H. Papadimitriou, *Computational Complexity*, Addison-Wesley, 1994.
- [6] L. J. Stockmeyer und A. R. Meyer, *Word problems requiring exponential time*, Proceedings of the 5th Annual ACM Symposium on Theory of Computing (STOC), S. 1–9, 1973.
- [7] T. Baker, J. Gill und R. Solovay, *Relativizations of the $P=NP$ question*, SIAM Journal on Computing, 4(4):431–442, 1975.
- [8] A. A. Razborov und S. Rudich, *Natural proofs*, Journal of Computer and System Sciences, 55(1):24–35, 1997.
- [9] W. J. Masek und M. S. Paterson, *A faster algorithm computing string edit distances*, Journal of Computer and System Sciences, 20(1):18–31, 1980.
- [10] R. G. Downey und M. R. Fellows, *Parameterized Complexity*, Springer, 1999.
- [11] L. G. Valiant, *The complexity of computing the permanent*, Theoretical Computer Science, 8(2):189–201, 1979.
- [12] R. Impagliazzo und A. Wigderson, *$P = BPP$ if E requires exponential circuits: Derandomizing the XOR Lemma*, Proceedings of the 29th Annual ACM Symposium on Theory of Computing (STOC), S. 220–229, 1997.
- [13] L. K. Grover, *A fast quantum mechanical algorithm for database search*, Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC), S. 212–219, 1996.
- [14] J. Watrous, *PSPACE has constant-round quantum interactive proof systems*, Theoretical Computer Science, 292(3):575–588, 2003.
- [15] R. Sharan und T. Ideker, *Modeling cellular machinery through biological network comparison*, Nature Biotechnology, 24(4):427–433, 2006.
- [16] E. M. Clarke, O. Grumberg und D. A. Peled, *Model Checking*, MIT Press, 1999.

- [17] K. D. Mulmuley und M. Sohoni, *Geometric complexity theory I: An approach to the P vs. NP and related problems*, SIAM Journal on Computing, 31(2):496–526, 2001.
- [18] N. Immerman, *Descriptive Complexity*, Springer, 1999.
- [19] R. Williams, *Faster all-pairs shortest paths via circuit complexity*, Proceedings of STOC 2014, S. 664–673, 2014.
- [20] M. Furst, J. B. Saxe und M. Sipser, *Parity, circuits, and the polynomial-time hierarchy*, Mathematical Systems Theory, 17(1):13–27, 1984.
- [21] R. E. Ladner, *On the structure of polynomial time reducibility*, Journal of the ACM, 22(1):155–171, 1975.
- [22] N. Robertson und P. D. Seymour, *Graph minors. V. Excluding a planar graph*, Journal of Combinatorial Theory, Series B, 41(1):92–114, 1986.
- [23] N. Robertson und P. D. Seymour, *Graph minors. XIII. The disjoint paths problem*, Journal of Combinatorial Theory, Series B, 63(1):65–110, 1995.
- [24] B. Courcelle, *The monadic second-order logic of graphs. I. Recognizable sets of finite graphs*, Information and Computation, 85(1):12–75, 1990.
- [25] L. Babai, *Graph isomorphism in quasipolynomial time*, Proceedings of the 48th Annual ACM Symposium on Theory of Computing (STOC), S. 684–697, 2016.
- [26] L. Mancinska und D. E. Roberson, *Quantum homomorphisms*, Journal of Combinatorial Theory, Series B, 134:170–212, 2019.